

Tensorflow II

CNN Regularization

Fatemeh Dehrouyeh



- 1 CNN Regularization
 - Regularization and Better Training
 - MNIST Dataset

Current Section

1 CNN Regularization

- Regularization and Better Training

Current Subsection

- 1 CNN Regularization
 - Regularization and Better Training
 - MNIST Dataset

Why Regularization?

CNNs can easily memorize training images if the model is too flexible.

Regularization means using techniques that help the model generalize better.

- reduce overfitting,
- improve validation performance,
- make the model more robust to unseen images.

Goal

We want the model to learn useful visual patterns, not memorize the training set.

Dropout

Dropout randomly disables some neurons during training.

This forces the network to avoid relying too much on a small set of features.

Dropout (0.5)

means that approximately 50% of selected activations are dropped during training.

Important

Dropout is active during training, but inactive during inference.

Code

```

import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(128, 128, 3)),

    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.GlobalAveragePooling2D(),

    tf.keras.layers.Dropout(0.5),

    tf.keras.layers.Dense(10, activation="softmax")
])

model.summary()

```

... Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_1 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 64)	0
dropout (Dropout)	(None, 64)	0
dense (Dense)	(None, 10)	650

Total params: 20,042 (78.29 KB)

Trainable params: 20,042 (78.29 KB)

Non-trainable params: 0 (0.00 B)

Note

Dropout can also be placed after a convolutional layer to regularize the feature maps. For CNNs, `SpatialDropout2D` is often useful because it drops entire feature-map channels rather than individual values within the feature maps.

```
Conv2D(64, 3, padding="same", activation="relu"),  
SpatialDropout2D(0.3),  
MaxPooling2D(),
```

Batch Normalization

Batch normalization normalizes intermediate activations inside the network.

It can help training become:

- more stable,
- faster,
- less sensitive to initialization.

Common Pattern

Conv2D $\rightarrow$ BatchNorm $\rightarrow$ ReLU

Code

```
▶ model = tf.keras.Sequential([  
    tf.keras.layers.Input(shape=(128, 128, 3)),  
  
    tf.keras.layers.Conv2D(32, 3, padding="same", use_bias=False),  
    tf.keras.layers.BatchNormalization(),  
    tf.keras.layers.ReLU(),  
    tf.keras.layers.MaxPooling2D(),  
  
    tf.keras.layers.Conv2D(64, 3, padding="same", use_bias=False),  
    tf.keras.layers.BatchNormalization(),  
    tf.keras.layers.ReLU(),  
    tf.keras.layers.MaxPooling2D(),  
  
    tf.keras.layers.GlobalAveragePooling2D(),  
    tf.keras.layers.Dense(10, activation="softmax")  
)  
  
model.summary()
```

Code—Result

... Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 128, 128, 32)	864
batch_normalization (BatchNormalization)	(None, 128, 128, 32)	128
re_lu (ReLU)	(None, 128, 128, 32)	0
max_pooling2d_2 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_3 (Conv2D)	(None, 64, 64, 64)	18,432
batch_normalization_1 (BatchNormalization)	(None, 64, 64, 64)	256
re_lu_1 (ReLU)	(None, 64, 64, 64)	0
max_pooling2d_3 (MaxPooling2D)	(None, 32, 32, 64)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 64)	0
dense_1 (Dense)	(None, 10)	650

Total params: 20,330 (79.41 KB)

Trainable params: 20,138 (78.66 KB)

Non-trainable params: 192 (768.00 B)

Batch Normalization in Our CNN

Consider the first convolutional block in our model:

Model

$\text{Conv2D}(32) \rightarrow \text{BatchNorm} \rightarrow \text{ReLU}$

Assume that during training:

batch size = 32

The input batch has shape:

$(32, 128, 128, 3)$

After:

$\text{Conv2D}(32, 3, \text{padding}=\text{"same"})$

the output has shape:

$(32, 128, 128, 32)$

- 32 images in the batch
- each feature map is 128×128
- 32 convolutional feature channels

What Does BatchNorm Average Over?

For each feature channel, Batch Normalization calculates one mean and one variance.
For **feature channel 1**:

$$\mu_1 = \frac{1}{32 \cdot 128 \cdot 128} \sum_{n=1}^{32} \sum_{h=1}^{128} \sum_{w=1}^{128} x_{n,h,w,1}$$

This means the mean is calculated from:

$$32 \times 128 \times 128 = \boxed{524,288 \text{ activation values}}$$

- all 32 images in the current batch,
- all 128 rows,
- all 128 columns,
- but only feature channel 1.

The same calculation is performed independently for channels 2, 3, ..., 32.

What Happens to Each Activation?

For channel c , BatchNorm first calculates:

$$\mu_c \quad \text{and} \quad \sigma_c^2$$

using all images and all spatial positions in that channel.
Each activation is then normalized:

$$\hat{x}_{n,h,w,c} = \frac{x_{n,h,w,c} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}$$

BatchNorm then applies two learned parameters:

$$y_{n,h,w,c} = \gamma_c \hat{x}_{n,h,w,c} + \beta_c$$

Therefore, every location in the same channel uses the **same** μ_c , σ_c^2 , γ_c , and β_c .

Important

BatchNorm does **not** calculate a separate mean for every pixel position.

Why BatchNorm Comes Before ReLU

Our code performs:

$$\text{Conv2D} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU}$$

The convolution first produces raw activation values:

$$x_{n,h,w,c}$$

BatchNorm normalizes and rescales them:

$$x \rightarrow \hat{x} \rightarrow \gamma \hat{x} + \beta$$

Then ReLU is applied:

$$\text{ReLU}(y) = \max(0, y)$$

Therefore, BatchNorm operates on the convolution's **raw output before ReLU removes negative values**.

Gamma and Beta in Batch Normalization

After normalization:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Batch Normalization applies:

$$y = \gamma \hat{x} + \beta$$

- γ controls the **scale**.
- β controls the **shift**.
- Both are **learned during training**.

Why are they needed?

Because normalization alone would force every channel into a standardized distribution. Gamma and beta let the network decide how much of that normalization it actually wants to keep. BatchNorm learns a separate gamma and beta for each feature channel.

BatchNorm: How Many Parameters?

For a convolutional layer with C output channels, BatchNorm keeps four values for each channel:

$$\gamma_c, \quad \beta_c, \quad \mu_c, \quad \sigma_c^2$$

Therefore:

$$\text{Total BatchNorm parameters} = 4C$$

However, only γ and β are learned through backpropagation:

$$\text{Trainable parameters} = 2C$$

The moving mean μ and moving variance σ^2 are updated during training, but are not trainable:

$$\text{Non-trainable parameters} = 2C$$

Example

If the convolutional layer has 32 channels, $4 \times 32 = 128$ total parameters.

64 trainable + 64 non-trainable

Early Stopping

Early stopping stops training when validation performance stops improving.

This helps prevent the model from continuing to memorize the training data.

Idea

Stop training near the best validation performance, not necessarily after the last epoch.

Current Subsubsection

- 1 CNN Regularization
 - Regularization and Better Training
 - MNIST Dataset

What is MNIST?

- MNIST is a classic benchmark dataset for handwritten digit recognition.
- It contains grayscale images of digits from 0 to 9.
- Each image has size 28×28 pixels.
- It is commonly used to test machine learning and deep learning models.

Dataset Structure

- Training set: 60,000 images
- Test set: 10,000 images
- Number of classes: 10
- Image shape: $28 \times 28 \times 1$
- Pixel values are normalized from $[0, 255]$ to $[0, 1]$

Example MNIST Images

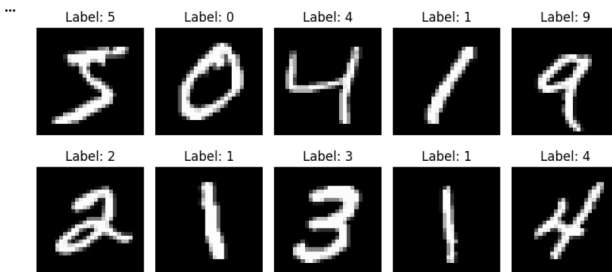
```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

plt.figure(figsize=(8, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i], cmap="gray")
    plt.title(f"Label: {y_train[i]}")
    plt.axis("off")

plt.tight_layout()
plt.savefig("mnist_samples.png", dpi=300)
plt.show()
```



Creating the Model

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

# Normalize the images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Add channel dimension: (28, 28) -> (28, 28, 1)
x_train = x_train[..., None]
x_test = x_test[..., None]

# Create training and validation datasets
train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train))
train_ds = train_ds.shuffle(60000).batch(32)

val_ds = tf.data.Dataset.from_tensor_slices((x_test, y_test))
val_ds = val_ds.batch(32)
```

Creating the Model—Continued

```
# Model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),

    tf.keras.layers.Conv2D(32, 3, padding="same", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(64, 3, padding="same", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
... Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ————— 0s 0us/step
```

Training the Model

```

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=30,
    callbacks=[early_stopping]
)

```

```

... Epoch 1/30
1875/1875 ————— 104s 54ms/step - accuracy: 0.7936 - loss: 0.8765 - val_accuracy: 0.9081 - val_loss: 0.4437
Epoch 2/30
1875/1875 ————— 142s 54ms/step - accuracy: 0.9233 - loss: 0.3360 - val_accuracy: 0.9297 - val_loss: 0.2692
Epoch 3/30
1875/1875 ————— 102s 54ms/step - accuracy: 0.9451 - loss: 0.2284 - val_accuracy: 0.9337 - val_loss: 0.2290
Epoch 4/30
1875/1875 ————— 143s 55ms/step - accuracy: 0.9547 - loss: 0.1809 - val_accuracy: 0.9428 - val_loss: 0.2105
Epoch 5/30
1875/1875 ————— 141s 54ms/step - accuracy: 0.9607 - loss: 0.1540 - val_accuracy: 0.9256 - val_loss: 0.2294
Epoch 6/30
1875/1875 ————— 101s 54ms/step - accuracy: 0.9655 - loss: 0.1332 - val_accuracy: 0.9616 - val_loss: 0.1492
Epoch 7/30
1875/1875 ————— 102s 54ms/step - accuracy: 0.9688 - loss: 0.1183 - val_accuracy: 0.9632 - val_loss: 0.1243
Epoch 8/30
1875/1875 ————— 101s 54ms/step - accuracy: 0.9710 - loss: 0.1081 - val_accuracy: 0.9545 - val_loss: 0.1615
Epoch 9/30
1875/1875 ————— 103s 55ms/step - accuracy: 0.9731 - loss: 0.0999 - val_accuracy: 0.9144 - val_loss: 0.2807
Epoch 10/30
1875/1875 ————— 101s 54ms/step - accuracy: 0.9750 - loss: 0.0924 - val_accuracy: 0.9605 - val_loss: 0.1367

```

Data Augmentation Inside a CNN

Data augmentation can be included directly inside the TensorFlow model.

This makes augmentation part of the training pipeline.

Important

Augmentation layers behave differently during training and inference. They modify images during training, but not during prediction.

Data Augmentation for 128×128 RGB Images

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1),
])

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(128, 128, 3)),

    data_augmentation,

    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.3),

    tf.keras.layers.Dense(10, activation="softmax")
])

model.summary()
```

Code—Result

... Model: "sequential_4"

Layer (type)	Output Shape	Param #
sequential_3 (Sequential)	(None, 128, 128, 3)	0
conv2d_6 (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d_6 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_7 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 32, 32, 64)	0
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None, 64)	0
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 10)	650

Total params: 20,042 (78.29 KB)

Trainable params: 20,042 (78.29 KB)

Non-trainable params: 0 (0.00 B)

A Better Small CNN

Now we can combine several good practices:

- data augmentation,
- convolution blocks,
- batch normalization,
- pooling,
- global average pooling,
- dropout,
- softmax output.

Goal

Build a small CNN that is simple, trainable, and less likely to overfit.

Code

```
num_classes = 10

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(128, 128, 3)),

    data_augmentation,

    tf.keras.layers.Conv2D(32, 3, padding="same", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(64, 3, padding="same", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(128, 3, padding="same", use_bias=False),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.ReLU(),

    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.4),

    tf.keras.layers.Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Code—Result

... Model: "sequential_5"

Layer (type)	Output Shape	Param #
sequential_3 (Sequential)	(None, 128, 128, 3)	0
conv2d_8 (Conv2D)	(None, 128, 128, 32)	864
batch_normalization_4 (BatchNormalization)	(None, 128, 128, 32)	128
re_lu_4 (ReLU)	(None, 128, 128, 32)	0
max_pooling2d_8 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_9 (Conv2D)	(None, 64, 64, 64)	18,432
batch_normalization_5 (BatchNormalization)	(None, 64, 64, 64)	256
re_lu_5 (ReLU)	(None, 64, 64, 64)	0
max_pooling2d_9 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_10 (Conv2D)	(None, 32, 32, 128)	73,728
batch_normalization_6 (BatchNormalization)	(None, 32, 32, 128)	512
re_lu_6 (ReLU)	(None, 32, 32, 128)	0
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None, 128)	0
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 10)	1,290

Total params: 95,210 (371.91 KB)

Trainable params: 94,762 (370.16 KB)

Non-trainable params: 448 (1.75 KB)

Data Augmentation for MNIST

If we want to use data augmentation with MNIST, we generally should not horizontally flip handwritten digits because this can create unrealistic or misleading shapes. Small rotations and zooms are more appropriate.

A suitable augmentation approach for MNIST is:

```
import tensorflow as tf

data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1),
])

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),

    data_augmentation,

    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dropout(0.3),

    tf.keras.layers.Dense(10, activation="softmax")
])
```

Congratulations!

Now you know about CNN Regularization!

Modern computer vision often uses pretrained models and transfer learning instead of training CNNs from scratch.

